

## **Activity 03: Evaluating and Debugging Code Using AI**

Sharie Rhea

Southern New Hampshire University

CS 530: Artificial Intelligence Principles and Applications

Joshua Andrews

February 22, 2026

Many modern integrated development environments (IDEs) now incorporate artificial intelligence (AI) tools to assist with coding, debugging, and documentation tasks. One such tool is GitHub Copilot, which is available as an extension for Visual Studio Code. In this activity, Copilot was used to evaluate, debug, and optimize a small Python file, `snippet.py`. This activity first assesses Copilot's ability to identify issues in the code, then examines the efficiency of using AI for debugging and evaluation, and finally considers the broader utility of AI for programming tasks.

In practice, Copilot identified not only all of the listed issues in the code snippet but also several additional undocumented ones. In response to a single short prompt, Copilot generated a list of 26 code issues. Each issue included a brief explanation, the corresponding line number, and a link to its inline reference in the file. For example, it flagged errors such as “Line 7: Typo in variable name numbrs (should be numbers)” and “Line 78: Reserved keyword class used as variable name” (GitHub, 2026). Beyond identifying explicit bugs, the tool also highlighted inefficient sections of the code, such as using two separate for loops for summing the numbers and finding the maximum, as well as redundant variable assignments (GitHub, 2026). Finally, Copilot improved the readability of the code by including documentation and inline comments (GitHub, 2026).

The primary motivation for using AI in code evaluation and debugging is improved efficiency. If prompting the AI requires excessive effort to achieve the desired result, a programmer would likely be better off fixing the code manually. Copilot demonstrates clear strengths when debugging a simple script such as this one. Because the task involves only a single file, the tool can analyze the full context and more easily infer the goal of the code. This is why Copilot was

able to easily determine the main function of the file: “The code attempts to calculate statistical measures (average, maximum, and minimum) from a list of numbers” (GitHub, 2026). Speed represents another significant advantage. Copilot was able to list 26 code issues within a few seconds, while a programmer would have to first parse the code, understand its goals, and individually examine each potential bug. Additionally, Copilot was able to improve the efficiency of the code itself by combining tasks and removing unnecessary code.

Despite these advantages, relying on AI for debugging also presents notable drawbacks. First, it is difficult to verify that the resulting code has the desired functionality without a set of external tests. In this particular example, the `snippet.py` file had a lot of code that did not produce a tangible result, so Copilot removed it. Although this technically resolves the identified bugs, it does not demonstrate the tool’s ability to handle more complex issues, such as bare exceptions or type errors (GitHub, 2026). It should be noted that Copilot did correctly identify these bugs, despite many of them residing in a section of unreachable code following the return statement. This is a strong indication that Copilot would be capable of resolving many types of errors in files that are quite syntactically “messy.” Scalability presents another significant limitation of using AI for code evaluation and debugging. Copilot worked wonderfully on the small file used in this example, but would likely struggle when a project scales up to hundreds of files with thousands of lines of code. Generative pretrained transformers are only capable of taking in so many tokens of input, and providing an entire project would essentially mean training a whole new model.

Overall, AI serves as a powerful tool for assisting programmers in evaluating and debugging code. For short, independent files, an agentic AI such as Copilot can increase programmer

efficiency by directly resolving bugs and inefficient code. These tools can also reduce the effort required for routine tasks such as writing documentation. Nevertheless, programmers should avoid blindly accepting all suggested changes. AI can make mistakes, and it is often difficult to externally validate AI-generated code. For simple tasks, such as working with a single function or class, AI is an efficient way to debug or improve code. For entire projects, it is likely that the amount of time spent attempting to refine the tool's responses will quickly outgrow the time necessary to achieve the desired result manually.

## References

GitHub. (2026). *Copilot*. <https://github.com/features/copilot/ai-code-editor>